

FLOYD-WARSHALL ALGORITHM

determines shortest path between all pairs of vertices in a graph, if there is no negative cycle

EXAMPLE

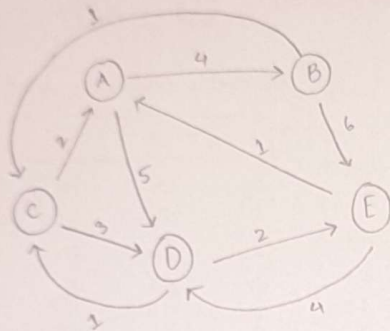
- initialize solution matrix same as the input graph matrix.
- consider all vertices as an intermediate vertex.

for $k=0$ to $n-1$

for $i=0$ to $n-1$

for $j=0$ to $n-1$

$$\text{Distance}[i,j] = \min(\text{Distance}[i,j], \text{Distance}[i,k] + \text{Distance}[k,j])$$



Step 1. Initialize $\rightarrow D^0$

	0	1	2	3	4	5
0		A	B	C	D	E
1	A	0	4	∞	5	∞
2	B	∞	0	1	∞	6
3	C	2	∞	0	3	∞
4	D	∞	∞	1	0	2
5	E	1	∞	∞	4	0

Step 2. treat node A as intermediate $\rightarrow D^1, k=1$

	0	1	2	3	4	5
0		A	B	C	D	E
1	A	0	4	∞	5	∞
2	B	∞	0	1	∞	6
3	C	2	6	0	3	∞
4	D	∞	∞	1	0	2
5	E	1	5	∞	4	0

$$\begin{aligned}
 D^1 &= \min(\text{dist}[2,3], \text{dist}[2,1] + [1,3]) = 1, \infty + \infty \\
 &= \min(\text{dist}[2,4], \text{dist}[2,1] + [1,4]) = \infty, \infty + 5 \\
 &= \min(\text{dist}[2,5], \text{dist}[2,1] + [1,5]) = 6, \infty + \infty \\
 &= \min(\text{dist}[3,2], [3,1] + [1,2]) = \infty, 2 + 4 \\
 &= \min(\text{dist}[3,4], [3,1] + [1,4]) = 3, 2 + 5 \\
 &= \min(\text{dist}[3,5], [3,1] + [1,5]) = \infty, 2 + \infty \\
 &= \min(\text{dist}[4,2], [4,1] + [1,2]) = \infty, \infty + 4 \\
 &= \min(\text{dist}[4,3], [4,1] + [1,3]) = 1, \infty + \infty \\
 &= \min(\text{dist}[4,5], [4,1] + [1,5]) = 2, \infty + \infty \\
 &= \min(\text{dist}[5,2], [5,1] + [1,2]) = \infty, 1 + 4 \\
 &= \min(\text{dist}[5,3], [5,1] + [1,3]) = \infty, 1 + \infty \\
 &= \min(\text{dist}[5,4], [5,1] + [1,4]) = 4, 1 + 5
 \end{aligned}$$

Step 3. treat node B as intermediate $\rightarrow D^2, k=2$

	0	1	2	3	4	5
0		A	B	C	D	E
1	A	0	4	5	5	10
2	B	∞	0	1	∞	6
3	C	2	6	0	3	12
4	D	∞	∞	1	0	2
5	E	1	5	6	4	0

$$\begin{aligned}
 [1,5] &= \infty & [1,2] + [2,3] &= 4 + 1 = 5 \\
 [1,4] &= 5 & [1,2] + [2,4] &= 4 + \infty = \infty \\
 [1,5] &= \infty & [1,2] + [2,5] &= 4 + 6 = 10 \\
 [3,2] &= 2 & [3,2] + [2,3] &= 6 + \infty = \infty \\
 [3,4] &= 3 & [3,2] + [2,4] &= 6 + \infty = \infty \\
 [3,5] &= \infty & [3,2] + [2,5] &= 6 + 6 = 12 \\
 [4,2] &= \infty & [4,2] + [2,3] &= \infty + \infty = \infty \\
 [4,3] &= 1 & [4,2] + [2,4] &= \infty + 1 = \infty \\
 [4,5] &= 2 & [4,2] + [2,5] &= \infty + 6 = \infty \\
 [5,1] &= 1 & [5,2] + [2,3] &= 5 + \infty = \infty \\
 [5,3] &= \infty & [5,2] + [2,4] &= 5 + 1 = 6 \\
 [5,4] &= 4 & [5,2] + [2,5] &= 5 + 6 = 11
 \end{aligned}$$

Step 4: treat node C as intermediate

	0	1	2	3	4	5
0		A	B	C	D	E
1 A	0		4	5	5	10
2 B	3	0		1	4	6
3 C	2	6	0	8	12	
4 D	3	7	1	0	2	
5 E	1	5	6	9	0	

$$[1,2] = 4$$

$$[1,4] = 5$$

$$[1,5] = 10$$

$$[2,1] = 3$$

$$[2,4] = 3$$

$$[2,5] = 6$$

$$[4,1] = 5$$

$$[4,2] = 3$$

$$[4,5] = 2$$

$$[5,1] = 1$$

$$[5,2] = 5$$

$$[5,4] = 4$$

$$[1,3] + [3,2] = 5 + 6 = 11$$

$$[1,3] + [3,4] = 5 + 3 = 8$$

$$[1,3] + [3,5] = 5 + 12 = 17$$

$$[2,3] + [3,1] = 1 + 2 = 3$$

$$[2,3] + [3,4] = 1 + 3 = 4$$

$$[2,3] + [3,5] = 1 + 12 = 13$$

$$[4,3] + [3,1] = 1 + 2 = 3$$

$$[4,3] + [3,2] = 1 + 6 = 7$$

$$[4,3] + [3,5] = 1 + 12 = 13$$

$$[5,3] + [3,1] = 6 + 2 = 8$$

$$[5,3] + [3,2] = 6 + 6 = 12$$

$$[5,3] + [3,4] = 6 + 3 = 9$$

Step 5: treat node D as intermediate $\rightarrow D^1, k=3$

	0	1	2	3	4	5
0		A	B	C	D	E
1 A	0		4	5	5	7
2 B	3	0		1	4	6
3 C	2	6	0	3	5	
4 D	3	7	1	0	2	
5 E	1	5	5	4	0	

$$[1,2] = 4$$

$$[1,3] = 5$$

$$[1,5] = 10$$

$$[2,1] = 3$$

$$[3,1] = 1$$

$$[2,5] = 6$$

$$[3,1] = 2$$

$$[3,2] = 6$$

$$[3,5] = 12$$

$$[5,1] = 1$$

$$[5,2] = 5$$

$$[5,3] = 6$$

$$[1,4] + [4,2] = 5 + 7 = 12$$

$$[1,4] + [4,3] = 5 + 1 = 6$$

$$[1,4] + [4,5] = 5 + 2 = 7$$

$$[2,4] + [4,1] = 4 + 3 = 7$$

$$[2,4] + [4,3] = 4 + 1 = 5$$

$$[2,4] + [4,5] = 4 + 2 = 6$$

$$[3,4] + [4,1] = 3 + 3 = 6$$

$$[3,4] + [4,2] = 3 + 7 = 10$$

$$[3,4] + [4,3] = 3 + 2 = 5$$

$$[5,4] + [4,1] = 4 + 3 = 7$$

$$[5,4] + [4,2] = 4 + 7 = 11$$

$$[5,4] + [4,3] = 4 + 1 = 5$$

Step 6: treat node E as intermediate

	0	1	2	3	4	5
0		A	B	C	D	E
1 A	0		4	5	5	7
2 B	3	0		1	4	6
3 C	2	6	0	3	5	
4 D	3	7	1	0	2	
5 E	1	5	5	4	0	

$$[1,2] = 4$$

$$[1,3] = 5$$

$$[1,4] = 5$$

$$[2,1] = 3$$

$$[2,3] = 1$$

$$[2,4] = 4$$

$$[3,1] = 2$$

$$[3,2] = 6$$

$$[3,4] = 3$$

$$[4,1] = 3$$

$$[4,2] = 7$$

$$[4,3] = 1$$

$$[1,5] + [5,2] = 7 + 5 = 12$$

$$[1,5] + [5,3] = 7 + 5 = 12$$

$$[1,5] + [5,4] = 7 + 4 = 11$$

$$[2,5] + [5,1] = 6 + 1 = 7$$

$$[2,5] + [5,3] = 6 + 5 = 11$$

$$[2,5] + [5,4] = 6 + 0 = 6$$

$$[3,5] + [5,1] = 5 + 1 = 6$$

$$[3,5] + [5,2] = 5 + 5 = 10$$

$$[3,5] + [5,4] = 5 + 4 = 9$$

$$[4,5] + [5,1] = 2 + 1 = 3$$

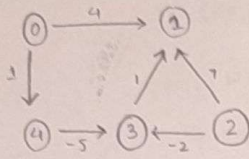
$$[4,5] + [5,2] = 2 + 5 = 7$$

$$[4,5] + [5,3] = 2 + 5 = 7$$

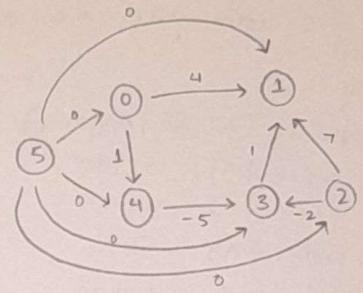
JOHNSON'S ALGORITHM

- add a new vertex S to the graph, and add edges from the new vertex of G (graph).
- run bellman ford with S as the source. the negative weight cycle cannot be created by new vertex S as there is no edge to S . All edges are from S .
- reweight the edges of the original graph. For each edge (u,v) assign the new weight as $\text{original} + h[u] - h[v]$
- remove added vertex and run dijkstra.

1. original



2. after adding vertex $S=5$



3. run bellman ford for $v-1$ iteration i.e. $6-1=5$

	0	1	2	3	4
1st iteration	∞	∞	∞	∞	∞
	0	0	0	0	0
	0	0	0	-2	0
	0	-1	0	-2	0
	0	-1	0	-5	0

5-0	0-1
5-1	0-4
5-2	2-1
5-3	2-3
5-4	3-1
	1

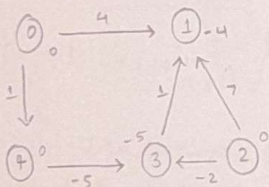
4-3

$$d[u] + w(u,v) \neq d[v]$$

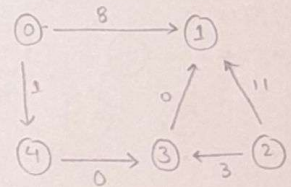
2nd iteration	0	-1	0	-5	0
	0	-4	0	-5	0

3rd, 4th, 5th iteration remains same.

4. applying $w(u,v) = d[u] + w(u,v) - d[v]$



$$\begin{aligned} 0 \rightarrow 1 &: 0 + 4 - (-4) = 8 \\ 2 \rightarrow 1 &: 0 + 7 - (-4) = 11 \\ 4 \rightarrow 3 &: 0 - 5 - (-5) = 0 \\ 0 \rightarrow 4 &: 0 + 1 - 0 = 1 \\ 3 \rightarrow 1 &: -5 + 1 - (-4) = 0 \\ 2 \rightarrow 3 &: 0 - 2 - (-5) = 3 \end{aligned}$$



5. applying dijkstra's

Vertex	Distance	Parent
✓ 0	0	∅ null
✓ 1	8	0
2	∞	∅ unreachable
✓ 3	1	4
✓ 4	1	0

Vertex	Distance	Parent
0	∞	null
✓ 1	8	3
✓ 2	0	null
✓ 3	3	2
4	∞	null

Vertex	Distance	Parent
0	∞	
1	0	∅ null
2	∞	
3	0	∅ unreachable
4	0	

Vertex	distance	parent
0	∞	-
✓ 1	0	3
2	∞	-
✓ 3	0	null
4	∞	-

Vertex	distance	parent
0	∞	
1	0	3
2	∞	
3	0	4
4	0	-

MATRIX CHAIN MULTIPLICATION

1. Brute force approach.

if matrices are all square and same size, multiplication order will not affect the total cost.

if $A = a[i,j]_{p,q}$ and $B = b[i,j]_{q,r}$

then $C = AB$ and order of C is $p \times r$

C = total entries in $C \times$ cost of computing a single entry = $p \times r \times q$

example:

sequence $[A_1, A_2, A_3, A_4]$

order of $A_1 = 10 \times 100$

order of $A_2 = 100 \times 5$

order of $A_3 = 5 \times 50$

order of $A_4 = 50 \times 20$

possible ways $\rightarrow 1. (A_1 (A_2 (A_3 A_4)))$

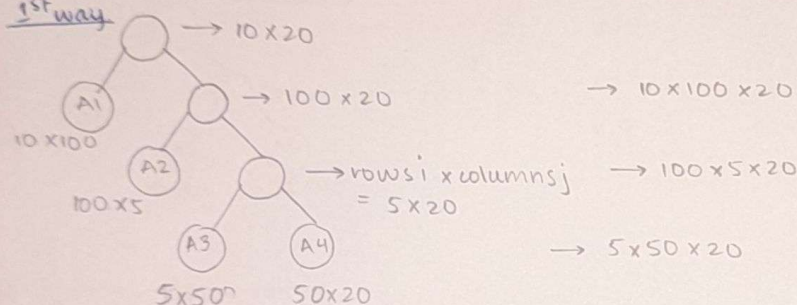
2. $(A_1 ((A_2 A_3) A_4))$

3. $((A_1 A_2) A_3) A_4$

4. $((A_1 A_2) (A_3 A_4))$

5. $((A_1 (A_2 A_3)) A_4)$

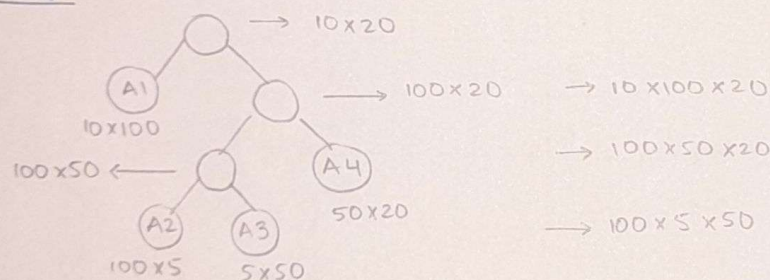
1st way



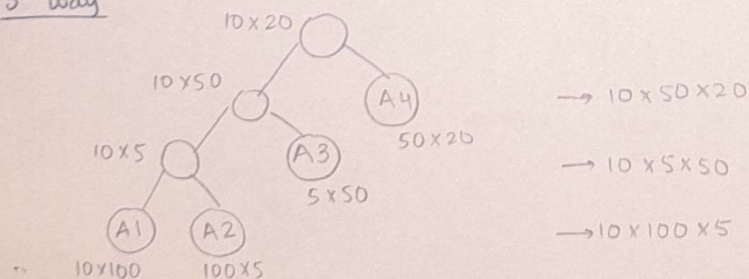
* for each result node take rows $\times$ columns

* for calculating minimum cost take rows $A_i \times$ columns $A_i \times$ rows A_j

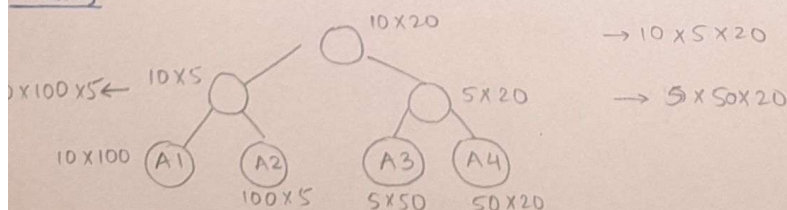
2nd way



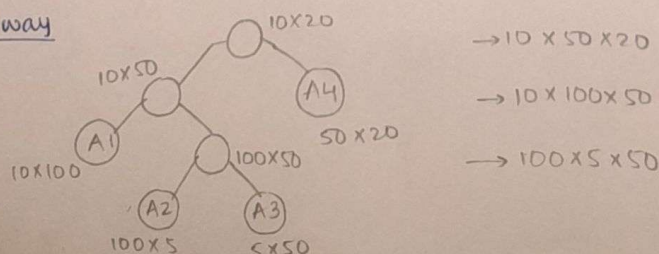
3rd way



4th way



5th way



Recursive Relation.

n = items i.e. matrices

$P(n)$ = total number of different ways.

$P(k)$ = number of ways to parenthesize left subsequence of k matrices.

$P(n-k)$ = right subsequence.

$$P(n) = \begin{cases} 1 & n=1 \\ \sum_{k=1}^{n-1} P(k) \cdot P(n-k) & n \geq 2 \end{cases}$$

for $n=3$; split at $k=1$

$$(A_1) \times (A_2 A_3) = 1$$

split at $k=2$

$$(A_1 A_2) \times (A_3) = 1$$

$$\text{for } n=3 \rightarrow P(3) = P(1) \cdot P(2) + P(2) \cdot P(1) = 1 + 1 = 2.$$

$$T(n) = \frac{2n C_n}{n+1}$$

$$T(3) = \frac{2(3) C_3}{3+1} = \frac{6 C_3}{4} = 5$$

example:

order (A1, A2, A3)

A1: 10x20

A2: 20x30

A3: 30x40

possible ways $\rightarrow n=3, k=1$

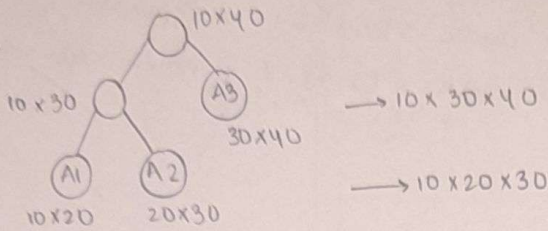
$$P(3) = P(1) * P(2) + P(2) * P(1)$$

$$= 1 + 1 = 2.$$

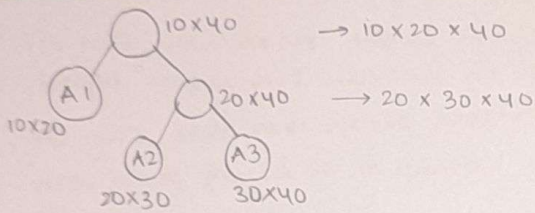
$$1. ((A1 \times A2) \times A3)$$

$$2. (A1 (A2 A3))$$

1st way



2nd way



2. Dynamic Programming.

m	1	2	3	4
1	0	120	88	158
2		0	48	104
3			0	84
4				0

$$A1: 10 \times 100 \rightarrow 5 \times 4$$

$$A2: 100 \times 5 \rightarrow 4 \times 6$$

$$A3: 5 \times 50 \rightarrow 6 \times 2$$

$$A4: 50 \times 20 \rightarrow 2 \times 7$$

$$m[1,2] = 5 \times 4 \times 6 \quad m[2,3] = 4 \times 6 \times 2 \quad m[3,4] = 6 \times 2 \times 7$$

$$m[1,3] = 5 \times 6 \times 2$$

$$m[2,4] = 4 \times 2 \times 7$$

$$m[1,4] =$$

$$\rightarrow ((A1 \ A2) \ A3)$$

$$\rightarrow ((A2 \ A3) \ A4)$$

$$m[1,2] + m[3,3] + 5 \times 6 \times 2$$

$$120 + 0 + 60$$

$$= 180$$

$$m[2,3] + m[4,4] + 4 \times 2 \times 7$$

$$= 48 + 0 + 56$$

$$= 104$$

$$(A1 \ (A2 \ A3))$$

$$(A2 \ (A3 \ A4))$$

$$m[1,1] + m[2,3] + 5 \times 4 \times 2$$

$$= 0 + 48 + 40$$

$$= 88$$

$$m[2,2] + m[2,3] + 6 \times 7 \times 4$$

$$= 0 + 48 + 168$$

$$= 216$$

$$\min[i,j] = \min_{i \leq k < j} (m[i,k] + m[k+1,j] + p_{i-1} \cdot p_k \cdot p_j)$$

$$\min[1,4] = \min(m[1,1] + m[2,4] + 5 \times 4 \times 7, m[1,2] + m[3,4] + 5 \times 6 \times 7, m[1,3] + m[4,4] + 5 \times 2 \times 7)$$

$$= 0 + 104 + 140 \quad \downarrow \quad = 120 + 84 + 210 \quad = 88 + 0 + 70$$

$$= 244 \quad \downarrow \quad = 414 \quad = 158$$

$$4 \times 2$$

$$4 \times 7$$

$$5 \times 4 \times 7$$

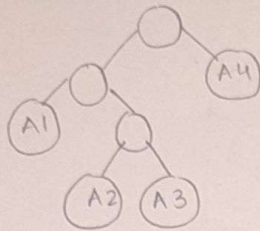
in the min formula, we consider $i \leq k < j$

so if we have $\min(1 \leq k < 2) \rightarrow$ in sequence table we write $k=1$

if we have $\min(1 \leq k < 3) \rightarrow$ we have 2 option $k=1$ and $k=2$, whichever is giving minimum.

now from sequence table we can get order of parenthesis;

$((A1)(A2 A3))(A4)$



- time complexity $O(n^3)$

★ start from top rightmost corner of sequence table and place bracket after 3

★ now check $S[1,3] = 1$ so separate 1

★ now check $S[2,3] = 2$ so no change.

0/1 KNAPSACK PROBLEM

1. dynamic approach.

example.

maximum capacity = 8

number of items = 4

$P = \text{profit} \{1, 2, 5, 6\}$

$W = \text{weight} \{2, 3, 4, 5\}$

steps for creating table.

1. create $n+1$ rows
2. create $w+1$ columns.

		0	1	2	3	4	5	6	7	8
P	W	0	0	0	0	0	0	0	0	0
1	2	1	0	0	1	1	1	1	1	1
2	3	2	0	0	1	2	2	3	3	3
5	4	3	0	0	1	2	5	5	6	7
6	5	4	0	0	1	2	5	6	7	8

$$T(i, j) = \max \{ T(i-1, j), \text{value}_j + T(i-1, j - \text{weight}_i) \}$$

- For row 1, since we are talking about only single item so add the profit on its corresponding weight so when $w=2$ can only fill when capacity is 2.
- For row 2, since now we are taking 2 items so when capacity is 3, only then second item can be placed so in column 3, place the profit that is 2 and before that copy values from above row.
- For row 2, $T[2, 5]$, both the items weight are $2+3=5$ which can be filled in column 5 so add their profits that is $1+2=3$ and write it in $T[2, 5]$. now both items are considered, fill columns after that with 3 only and $T[2, 4]$ will be 2.
- For row 3, consider items 2 and 3, their weight = 7 and profit = 7 so write in column 7 and for items 1 and 3 weight = 6 and profit = 6 so replace in column 6.
- For row 4,

$$T(4, 1) = \max \{ T(3, 1), 6 + T(3, 1-3) \} = 0$$

$$T(4, 6) = \max \{ T(3, 6), 6 + T(3, 6-5) \} = 6, 6+0 = 6$$

$$T(4, 7) = \max \{ T(3, 7), 6 + T(3, 7-5) \} = 7, 6+1 = 7$$

$$T(4, 8) = \max \{ T(3, 8), 6 + T(3, 8-5) \} = 7, 6+2 = 8$$

→ how to know which object should be included.

item 1	item 2	item 3	item 4
0	1	0	1

- starting from last row we have 8, which is not in the above row, hence included ~~in~~ 4th object, remaining profit is $8-6=2$.
- check if 2 is in 3rd row, it is. now check if it is in above row. it is. so do not include 3rd object.
- check if 2 is in 2nd row. it is. But not in the row above so include 2nd object as well. remaining profit is $2-2=0$.
- 0 is in 1st row and above that as well, hence do not include 1st object.

RECURRENCES

→ used to represent the running time of a recursive algorithm.

$$T(n) = \begin{cases} 1 & n \leq 1 \\ aT(\frac{n}{b}) + f(n) & n > 1 \end{cases}$$

METHODS

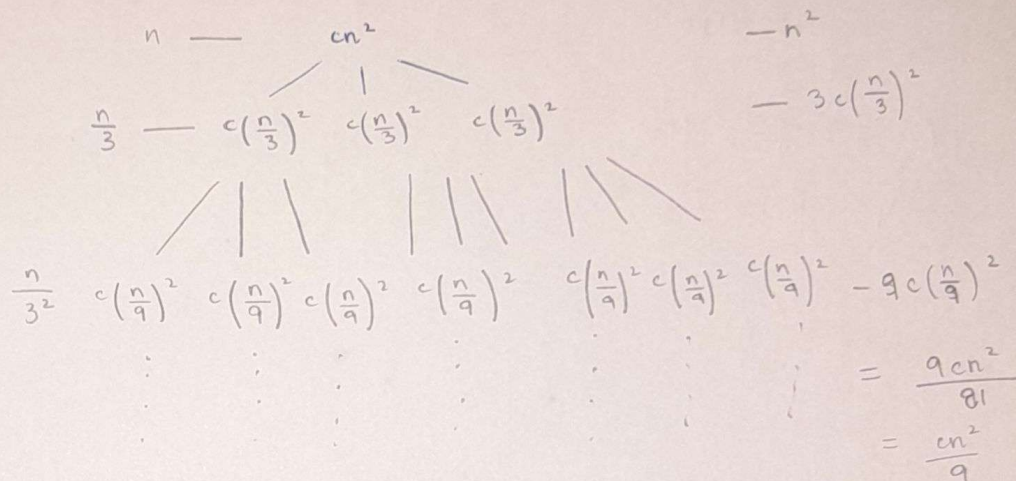
1. Iterative (Back substitution)
2. Recursion Tree
3. Master Method.

RECURSION TREE METHOD

$$T(n) = 3T(\frac{n}{3}) + cn^2$$

$$T(1) = c$$

- identify size on each level
- identify cost on each level



MASTER METHOD

case 1: $O(n^{\log_b a - \epsilon})$ where $\epsilon > 0$.

$$T(n) = O(n^{\log_b a})$$

case 2: $O(n^{\log_b a})$

$$T(n) = O(n^{\log_b a} \log n)$$

case 3: $\Omega(n^{\log_b a + \epsilon})$ and $a f(\frac{n}{b}) \leq c f(n)$

$$T(n) = O(f(n))$$

$$T(n) = a T\left(\frac{n}{b}\right) + f(n)$$

• compare $f(n)$ and $n^{\log_b a}$